

Task Block 2 - Multi-Agent System Report

Проект: Kopilkin - personal finance web application with AI assistant

Система: Kopilkin Multi-Agent Financial Assistant

Автор: Mubarakov Islam

Русская версия отчёта

1. Обзор проекта

Kopilkin - это mobile-first web application для личных финансов. Пользователь может зарегистрироваться, войти в аккаунт, добавлять доходы и расходы, создавать цели накоплений и задавать вопросы AI-ассистенту о своих тратах.

В рамках Task Block 2 проект был расширен до multi-agent AI system. Система помогает пользователю понять, куда уходят деньги, и даёт практические рекомендации по экономии на основе реальных данных из Transaction Service.

Главная идея: AI-ассистент не просто генерирует общий текст, а работает как связка нескольких специализированных агентов, подключённых к памяти, бизнес-сервисам, локальной LLM и системе observability.

2. Спецификация агента

Kopilkin agent system состоит из четырёх логических компонентов.

2.1 Orchestrator Agent

Orchestrator Agent координирует полный жизненный цикл запроса. Он получает сообщение пользователя, выполняет поиск по памяти, обогащает текущий запрос предыдущим контекстом, вызывает Router Agent, запускает нужную цепочку агентов и сохраняет итоговое взаимодействие обратно в память.

2.2 Router Agent

Router Agent определяет, какой путь должен обработать запрос:

- analyst - если пользователь спрашивает о числах, категориях, статистике, суммах и анализе трат;
- advisor - если пользователь просит советы, рекомендации, экономию или улучшение финансового поведения.

2.3 Analyst Agent

Analyst Agent получает реальные финансовые данные из Transaction Service и объясняет пользователю структуру расходов: общий доход, общие расходы, категории, самые большие траты и паттерны поведения.

2.4 Advisor Agent

Advisor Agent формирует практические рекомендации. Перед генерацией советов он использует результат Analyst Agent, поэтому советы основаны на реальных числах, а не на общих фразах.

3. Выбор LLM engine

Для локального запуска LLM были рассмотрены несколько вариантов runtime/engine.

Engine	Преимущества	Недостатки	Решение
Ollama	Простая установка, локальные модели, HTTP API, удобно для студенческого железа	Менее оптимизирован для high-load serving, чем vLLM	Выбран
llama.cpp	Очень лёгкий и эффективный локальный inference	Больше ручного управления моделями	Рассмотрен
vLLM	Высокопроизводительный serving больших моделей	Слишком тяжёлый для локального MVP	Не выбран

В качестве основного engine был выбран Ollama, потому что он быстро устанавливается локально, поддерживает много лёгких open-source моделей и предоставляет HTTP API, который удобно вызывать из FastAPI-сервисов.

4. Сравнение LLM моделей

Преподаватель не требовал конкретную LLM-модель. Требование заключалось в том, чтобы протестировать несколько lightweight local models, которые могут быть запущены на доступном железе, и аргументировать финальный выбор.

Для сравнения использовался один и тот же prompt:

Привет! Я трачу 5000 рублей в месяц на кафе и 3000 на транспорт. Что посоветуешь?

Критерий	llama3.2:3b	qwen2.5:3b	gemma3:4b
Понимание контекста	Поняла общий финансовый контекст	Слабое: ошибочно решила, что пользователь владеет кафе	Хорошее: поняла личные расходы
Русский язык	Слабый: смешение русского, английского и нестабильные фразы	Русский хороший, но контекст неверный	Русский хороший
Практическая полезность	Низкая: слишком общий и нестабильный ответ	Низкая: часть советов ушла в бизнес-менеджмент	Высокая: структурированный и релевантный ответ
Склонность к галлюцинациям	Есть	Есть	Минимальная в этом тесте
Итог	Не выбрана	Не выбрана	Выбрана

Финальный выбор: gemma3:4b.

Gemma 3 4B была выбрана как основная модель для Kopilkin multi-agent system, потому что она дала лучший баланс между качеством русского языка, пониманием контекста, практической пользой и низкой склонностью к галлюцинациям. В отличие от qwen2.5:3b, она не решила, что пользователь является владельцем кафе, а корректно поняла кафе и транспорт как категории личных расходов.

Сначала по сравнению моделей — вот итог:			
	llama3.2:3b	qwen2.5:3b	gemma3:4b
Понял контекст	✓	✗ (решил что ты владелец кафе)	✓
Русский язык	✗ (мешанина)	✓	✓
Полезность	слабо	слабо	хорошо
Галлюцинации	есть	есть	минимум
Победитель — gemma3:4b. Это и напишем в отчёте с реальными примерами.			

Рисунок 1. Скриншот ручного сравнения моделей.

5. Multi-agent architecture

Система не является набором независимых single agents. Агенты интегрированы в единый flow через routing и chaining.

User -> Frontend AI Chat -> Agent Service /chat

- > Orchestrator
- > Memory Search
- > Router Agent
- > Analyst Agent
- > Advisor Agent if advice is needed
- > Memory Save
- > Response

Advisor Agent зависит от результата Analyst Agent. Благодаря этому рекомендации по экономии grounded in real financial data, а не являются generic advice.

6. Диаграммы

Для Task Block 2 были добавлены новые диаграммы именно для agent system. Они находятся в папке Documentation/agent-diagrams/.

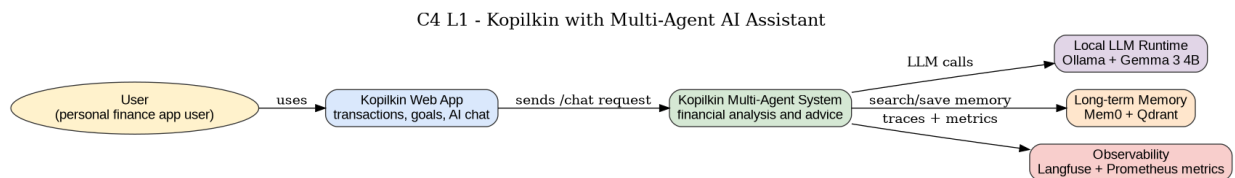


Рисунок 2. C4 L1 - Kopilkin with Multi-Agent AI Assistant.

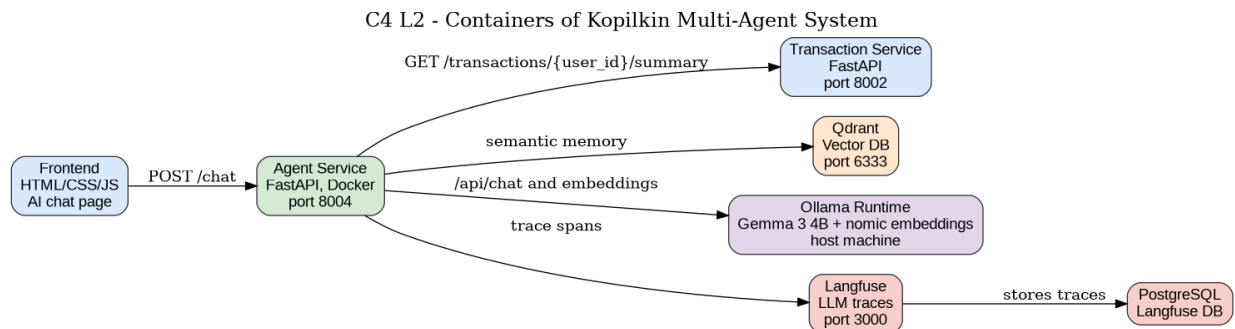


Рисунок 3. C4 L2 - Multi-Agent System Containers.

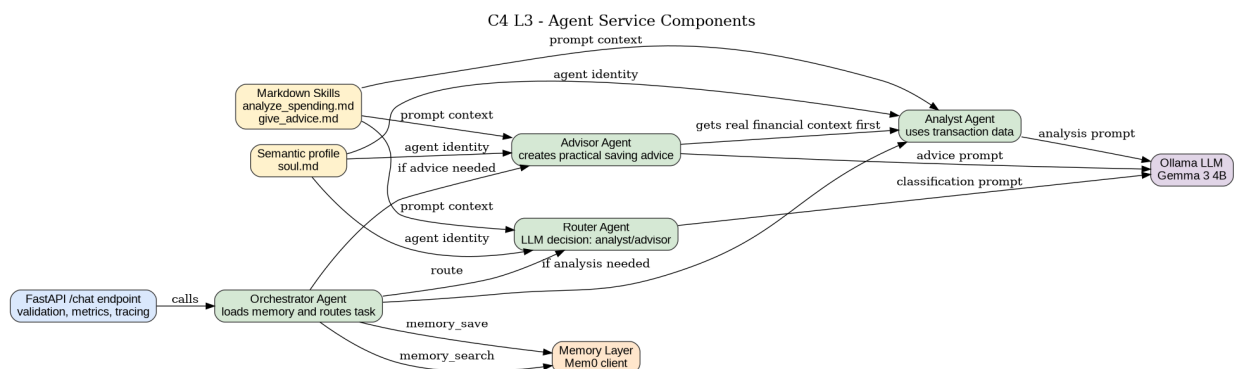


Рисунок 4. C4 L3 - Agent Service Components.

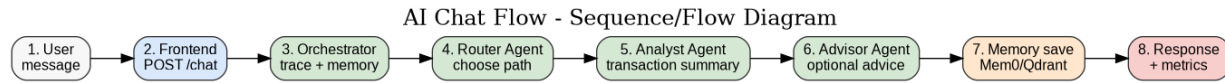


Рисунок 5. AI Chat Flow / Sequence Diagram.

7. Isolation strategy

Agent system развёрнут в Docker-based isolated environment.

Контейнеризированные компоненты:

- agent-service - FastAPI multi-agent service;
- qdrant - vector database для long-term memory;
- langfuse - LLM/agent tracing platform;
- langfuse-db - PostgreSQL database для Langfuse.

Ollama остаётся на host machine и доступен через host.docker.internal:11434. Это практичная hybrid setup, потому что локальный запуск модели зависит от runtime Ollama и ресурсов компьютера.

Вариант	Решение
No isolation	Отклонено: зависимости не воспроизводимы и среда небезопасна
Python venv	Недостаточно: изолирует только Python packages, но не базы и сервисы
Docker	Выбран: lightweight, reproducible, подходит для local MVP
Full VM	Слишком тяжело для моего ноута
Kubernetes	Future improvement, не нужен для локальной демонстрации
Sandbox runtime	Не выбран: агент не выполняет произвольный пользовательский код

8. System prompts

System prompts реализованы в файле:

```
agent-service/app/prompts/system_prompts.py
```

В проекте есть prompts для Orchestrator Agent, Analyst Agent и Advisor Agent. Они указывают агентам использовать реальные финансовые данные, избегать generic advice и отвечать на языке пользователя.

9. Skills и semantic markdown files

Skills описаны в Markdown-файлах:

```
agent-service/app/skills/analyze_spending.md
agent-service/app/skills/give_advice.md
```

Семантическая идентичность агента описана в файле:

```
agent-service/soul.md
```

Также был добавлен skill loader:

```
agent-service/app/skills/skill_loader.py
```

Skill loader читает .md skills и soul.md, чтобы использовать их внутри prompt context. Благодаря этому Markdown-файлы являются частью поведения агента, а не просто документацией.

10. Memory management system

Система использует Mem0 + Qdrant + Ollama embeddings.

- Mem0 управляет memory search и memory save;
- Qdrant хранит vector representations of memories;
- Ollama nomic-embed-text создаёт embeddings;
- Memory фильтруется по user_id.

10.1 Почему выбран Mem0

Kopilkin нужна personalized long-term memory, а не только поиск по статичным документам. Classic RAG лучше подходит для document question answering, но Kopilkin должен помнить повторяющиеся пользовательские проблемы, прошлые советы и контекст взаимодействий.

10.2 Типы памяти

- Short-term memory: текущий message, transaction summary, analyst result;
- Long-term memory: предыдущие interactions, сохранённые в Mem0/Qdrant.

10.3 Сравнение memory options

Option	Преимущества	Недостатки	Решение
Classic RAG	Простой retrieval из документов	Не оптимален для evolving personal memory и temporal user state	Не выбран как основной memory
Mem0	Простая user/session memory, подходит для personalization	Менее explainable, чем graph memory	Выбран
Zep	Conversation memory и fact extraction	Больше backend complexity	Рассмотрен
Letta / MemGPT	Advanced internal state и editable memory blocks	Слишком сложен для small MVP	Рассмотрен
Graphiti	Temporal knowledge graph и hybrid retrieval	Мощный, но heavier setup	Future improvement
LlamaIndex	Сильный data/RAG framework	Больше document-oriented	Рассмотрен

Главное ограничение текущего подхода: vector memory может вернуть семантически похожие, но логически не самые релевантные воспоминания. В production это можно улучшить через temporal graph memory или hybrid graph + vector architecture.

11. Evaluations

В проект добавлена папка evals:

evals/test_cases.json
evals/run_evals.py
evals/README.md

Metric	Meaning
Routing accuracy	Правильно ли Orchestrator выбирает Analyst/Advisor path
Groundedness	Использует ли ответ реальные transaction data
Hallucination rate	Не выдумывает ли модель несуществующие данные
Language consistency	Соответствует ли язык ответа языку пользователя

Tool success rate	Успешно ли вызывается Transaction Service
Memory relevance	Полезна ли найденная память
Latency	Сколько времени занимает /chat response
Error rate	Процент failed requests

Evaluation runner отправляет заранее заданные запросы в running agent service и сохраняет JSON report. Это базовый eval setup, который можно расширять через Langfuse datasets, automatic scoring и более строгие expected outputs.

12. Observability

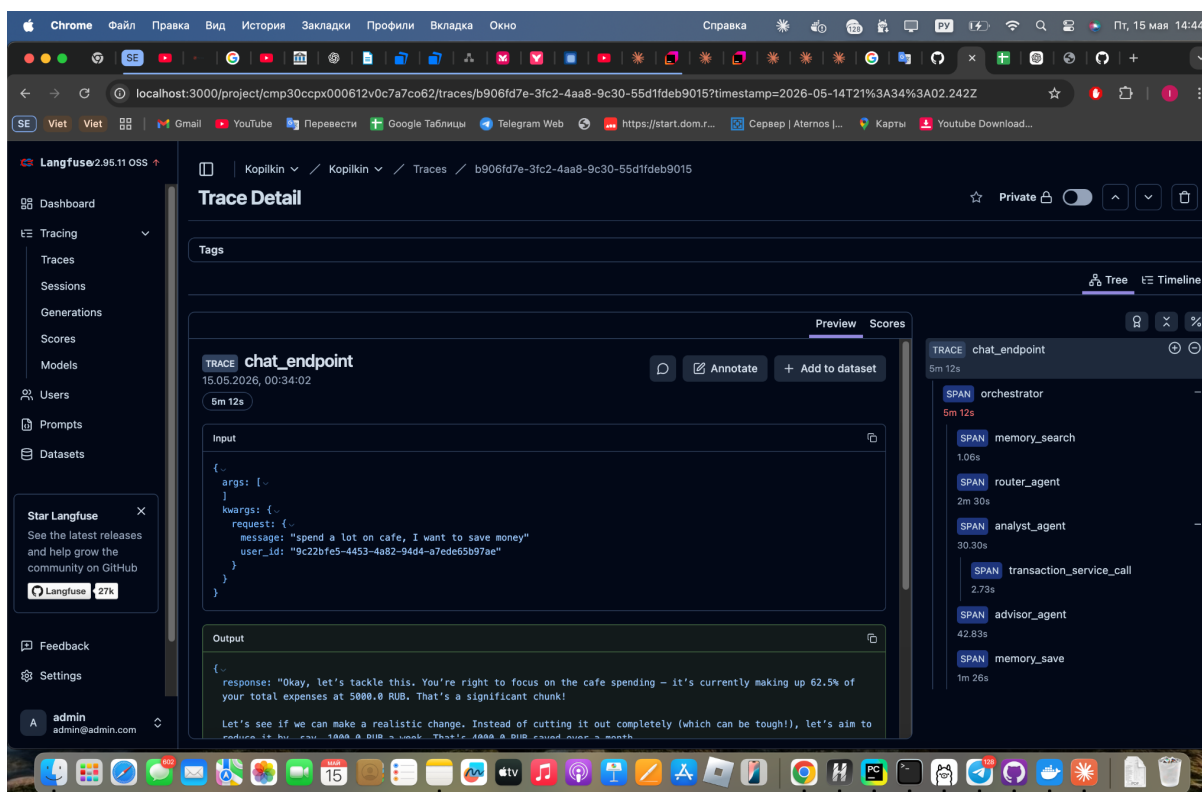
Implemented observability stack включает Langfuse и Prometheus-compatible metrics.

12.1 Langfuse traces

Langfuse трассирует основные этапы agent pipeline:

- chat_endpoint
- orchestrator
- memory_search
- router_agent
- analyst_agent
- transaction_service_call
- advisor_agent
- memory_save

Это позволяет увидеть полный путь генерации ответа: какой агент был вызван, где появилась latency, сработали ли memory/tool calls и на каком этапе возникла ошибка.



12.2 Metrics

Agent Service exposes:

GET /metrics

Implemented metrics:

- kopilkin_agent_chat_requests_total;
- kopilkin_agent_chat_errors_total;
- kopilkin_agent_chat_latency_seconds.

12.3 Logs

Agent Service пишет structured logs о начале запроса, завершении запроса, duration и errors.

12.4 Alerts

Full alerting запланирован как production improvement. Recommended alerts: high error rate, high latency, Ollama unavailable, Qdrant unavailable, Transaction Service unavailable, Langfuse unavailable.

13. Conclusion

Kopilkin теперь включает рабочую multi-agent AI system с local LLM execution, Docker-based isolation, long-term memory, semantic skills, system prompts, LLM comparison, evaluation files и observability через Langfuse и Prometheus-compatible metrics.

Основная модель - gemma3:4b, потому что она лучше всего показала себя в локальном сравнении: корректно поняла личные расходы, дала полезные советы на русском языке и показала минимальную склонность к hallucinations в тесте.

Текущая реализация подходит для MVP и oral defense. Для production improvement стоит добавить полноценный alerting, persistent databases для всех services, stronger secret management и более продвинутую систему memory evaluation.